Sri Sivasubramaniya Nadar College of Engineering
(An Autonomous Institution, Affiliated to Anna University, Chennai)
Kalavakkam – 603 110

Department of Computer Science & Engineering

Computer Networks Laboratory

1513 – Computer Networks Lab

Assignment 10

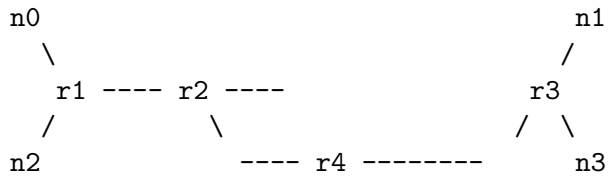
Comparative Evaluation of Routing Protocols Using NS-3

Name:	Sundareswaran Rajaguru
Register Number:	3122247001048
Year / Semester:	III Year / V Semester
Course:	1513 Computer Networks Lab
Date of Submission:	01 October 2026

1. Objective

To simulate a multi-router organizational network in NS-3 using three different routing approaches – RIP, OSPF, and AODV – and to compare their Packet Delivery Ratio, end-to-end delay, and routing-table update behaviour under normal traffic, congested traffic, and a mid-simulation link failure.

2. Topology



`n0-r1, n2-r1, r1-r2, r1-r4, r2-r3, r4-r3, r3-n1, r3-n3`

A diamond shape is used deliberately so that, when the r1–r2 link is disabled, an alternate path (r1–r4–r3) genuinely exists for a routing protocol to discover. `n0` sends CBR/UDP traffic to `n1` throughout; `n2`→`n3` is an additional flow used only in the congestion condition. All eight links run at 1 Mbps/2 ms.

Note on protocol fidelity. NS-3's mainline modules do not include an OSPF implementation, so `Ipv4GlobalRoutingHelper` – also a link-state, shortest-path algorithm, the same family OSPF belongs to – is used here as the closest available stand-in, not a claim of protocol-exact OSPF behaviour. AODV additionally needs a broadcast-capable link for its route-discovery flooding, so the AODV run alone builds the identical graph using `CsmaHelper` link segments instead of point-to-point links; the multi-hop structure does not change.

3. Algorithm

1. Build the eight-node diamond topology (4 hosts + 4 routers), using point-to-point links for RIP/OSPF and CSMA link segments for AODV.
2. Install the Internet stack with the selected routing protocol: `RipHelper` on the four routers (hosts get one static default route toward their attached router); `Ipv4GlobalRoutingHelper::PopulateRoutingTables()` for the OSPF stand-in; `AodvHelper` on every node for AODV.
3. Start a CBR/UDP (`OnOffHelper`, always-on, 512 B packets, 500 Kbps) flow from `n0` to `n1`. In the congestion condition, start an identical second flow `n2`→`n3` at the same time.
4. In the failure condition, bring down both ends of the r1–r2 interface at $t = \text{simTime}/2$ (`Ipv4::SetDown`); for the OSPF stand-in only, explicitly trigger `RecomputeRoutingTables()` shortly after, since global routing does not react to interface changes on its own.
5. Install `FlowMonitor`; run for 20 s; report Tx/Rx packet counts, PDR, and average delay per flow.
6. Enable each protocol's own `NS_LOG` output (`Rip` / `AodvRoutingProtocol`) and count control-plane send events as the "routing table updates" metric (see README for the exact log patterns matched).

4. Code

4.1 ass10.cc

```
// Assignment 10 - Comparative Evaluation of Routing Protocols in NS-3
// (RIP, "OSPF", AODV) under normal / congested / link-failure conditions.
//
// Topology (same graph for every protocol, a diamond so a failed link has an
// alternate path):
//
//        n0                                    n1
//          \                                  /
//           r1 ---- r2 ----                 r3
//          /          \                   /  \
//        n2             ---- r4 --------      n3
//
// Links: n0-r1, n2-r1, r1-r2, r1-r4, r2-r3, r4-r3, r3-n1, r3-n3
//
// n0 -> n1 is the primary CBR/UDP flow. In the "congestion" condition a
// second simultaneous flow n2 -> n3 is added. In the "failure" condition the
// r1-r2 link is brought down mid-simulation, forcing traffic via r1-r4-r3.
//
// --protocol=RIP|OSPF|AODV selects the routing approach:
//   RIP   - ns-3's built-in distance-vector RipHelper on the 4 routers.
//   OSPF  - ns-3 has no OSPF module; Ipv4GlobalRoutingHelper (a link-state
//           shortest-path algorithm, the same family OSPF belongs to) is
//           used as the closest available stand-in. This is a documented
//           simplification, not a real OSPF implementation.
//   AODV  - a genuine on-demand ad-hoc routing protocol. AODV needs a
//           broadcast-capable link for route discovery, so for this run
//           every link in the SAME graph is built with CSMA instead of
//           point-to-point (still one link = one 2-node segment, so the
//           multi-hop structure is identical either way).
//
// FlowMonitor gives Packet Delivery Ratio and end-to-end delay. "Routing
// table updates" are counted from each protocol's own NS_LOG control-message
// output (RIP route-update sends, AODV RREQ/RREP/RERR sends, or the number
// of global-routing recomputations for the OSPF stand-in).

#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/csma-module.h"
#include "ns3/applications-module.h"
#include "ns3/flow-monitor-module.h"
#include "ns3/aodv-module.h"

using namespace ns3;

NS_LOG_COMPONENT_DEFINE("RoutingCompare");

int
main(int argc, char *argv[])
{
    std::string protocol = "RIP";      // RIP | OSPF | AODV
    std::string condition = "normal"; // normal | congestion | failure
    double simTime = 20.0;

    CommandLine cmd;
    cmd.AddValue("protocol", "Routing approach: RIP, OSPF or AODV", protocol);
    cmd.AddValue("condition", "Network condition: normal, congestion or failure
        ", condition);
```

```
    cmd.AddValue("simTime", "Simulation duration (s)", simTime);
    cmd.Parse(argc, argv);

    if (protocol == "RIP") {
        LogComponentEnable("Rip", LOG_LEVEL_DEBUG);
    } else if (protocol == "AODV") {
        LogComponentEnable("AodvRoutingProtocol", LOG_LEVEL_LOGIC);
    }

    bool useCsma = (protocol == "AODV");

    NodeContainer hosts;   // 0=n0 1=n1 2=n2 3=n3
    hosts.Create(4);
    NodeContainer routers; // 0=r1 1=r2 2=r3 3=r4
    routers.Create(4);
    Ptr<Node> n0 = hosts.Get(0), n1 = hosts.Get(1), n2 = hosts.Get(2), n3 =
        hosts.Get(3);
    Ptr<Node> r1 = routers.Get(0), r2 = routers.Get(1), r3 = routers.Get(2), r4
         = routers.Get(3);

    // --- Internet stack + routing protocol selection ---
    InternetStackHelper hostStack;
    InternetStackHelper routerStack;
    if (protocol == "RIP") {
        RipHelper rip;
        routerStack.SetRoutingHelper(rip);
    } else if (protocol == "AODV") {
        AodvHelper aodv;
        hostStack.SetRoutingHelper(aodv);
        routerStack.SetRoutingHelper(aodv);
    }
    // OSPF stand-in and RIP hosts both use the default (global routing capable
        ) stack.
    hostStack.Install(hosts);
    routerStack.Install(routers);

    // --- Links: each "link" is one 2-node segment, CSMA for AODV else P2P ---
    auto makeLink = [&](Ptr<Node> a, Ptr<Node> b, Ipv4AddressHelper &addr) ->
        Ipv4InterfaceContainer {
        NetDeviceContainer dev;
        if (useCsma) {
            CsmaHelper csma;
            csma.SetChannelAttribute("DataRate", StringValue("1Mbps"));
            csma.SetChannelAttribute("Delay", StringValue("2ms"));
            dev = csma.Install(NodeContainer(a, b));
        } else {
            PointToPointHelper p2p;
            p2p.SetDeviceAttribute("DataRate", StringValue("1Mbps"));
            p2p.SetChannelAttribute("Delay", StringValue("2ms"));
            dev = p2p.Install(a, b);
        }
        Ipv4InterfaceContainer iface = addr.Assign(dev);
        addr.NewNetwork();
        return iface;
    };

    Ipv4AddressHelper addr;
    addr.SetBase("10.0.0.0", "255.255.255.0");
    makeLink(n0, r1, addr);
    makeLink(n2, r1, addr);
    Ipv4InterfaceContainer r1r2 = makeLink(r1, r2, addr);
    makeLink(r1, r4, addr);
    makeLink(r2, r3, addr);
```

```
    makeLink(r4, r3, addr);
    Ipv4InterfaceContainer r3n1 = makeLink(r3, n1, addr);
    Ipv4InterfaceContainer r3n3 = makeLink(r3, n3, addr);

    if (protocol == "OSPF") {
        Ipv4GlobalRoutingHelper::PopulateRoutingTables();
    }
    // RIP hosts (n0..n3) are not running RIP themselves; give them a default
    // route toward their attached router, as ns-3's own RIP examples do.
    if (protocol == "RIP") {
        Ipv4StaticRoutingHelper staticRouting;
        Ptr<Ipv4> ipv4;
        ipv4 = n0->GetObject<Ipv4>();
        staticRouting.GetStaticRouting(ipv4)->SetDefaultRoute(r1r2.GetAddress
            (0), 1);
        ipv4 = n2->GetObject<Ipv4>();
        staticRouting.GetStaticRouting(ipv4)->SetDefaultRoute(r1r2.GetAddress
            (0), 1);
        ipv4 = n1->GetObject<Ipv4>();
        staticRouting.GetStaticRouting(ipv4)->SetDefaultRoute(r3n1.GetAddress
            (0), 1);
        ipv4 = n3->GetObject<Ipv4>();
        staticRouting.GetStaticRouting(ipv4)->SetDefaultRoute(r3n3.GetAddress
            (0), 1);
    }

    // --- CBR/UDP traffic ---
    uint16_t port1 = 6000, port2 = 6001;
    PacketSinkHelper sink1("ns3::UdpSocketFactory", InetSocketAddress(
        Ipv4Address::GetAny(), port1));
    ApplicationContainer sinkApp1 = sink1.Install(n1);
    sinkApp1.Start(Seconds(0.0));
    sinkApp1.Stop(Seconds(simTime));

    OnOffHelper source1("ns3::UdpSocketFactory", InetSocketAddress(r3n1.
        GetAddress(1), port1));
    source1.SetAttribute("DataRate", StringValue("500Kbps"));
    source1.SetAttribute("PacketSize", UintegerValue(512));
    source1.SetAttribute("OnTime", StringValue("ns3::ConstantRandomVariable[
        Constant=100]"));
    source1.SetAttribute("OffTime", StringValue("ns3::ConstantRandomVariable[
        Constant=0]"));
    ApplicationContainer sourceApp1 = source1.Install(n0);
    sourceApp1.Start(Seconds(2.0));
    sourceApp1.Stop(Seconds(simTime));

    if (condition == "congestion") {
        PacketSinkHelper sink2("ns3::UdpSocketFactory", InetSocketAddress(
            Ipv4Address::GetAny(), port2));
        ApplicationContainer sinkApp2 = sink2.Install(n3);
        sinkApp2.Start(Seconds(0.0));
        sinkApp2.Stop(Seconds(simTime));

        OnOffHelper source2("ns3::UdpSocketFactory", InetSocketAddress(r3n3.
            GetAddress(1), port2));
        source2.SetAttribute("DataRate", StringValue("500Kbps"));
        source2.SetAttribute("PacketSize", UintegerValue(512));
        source2.SetAttribute("OnTime", StringValue("ns3::ConstantRandomVariable
            [Constant=100]"));
        source2.SetAttribute("OffTime", StringValue("ns3::
            ConstantRandomVariable[Constant=0]"));
        ApplicationContainer sourceApp2 = source2.Install(n2);
        sourceApp2.Start(Seconds(2.0));
```

```
        sourceApp2.Stop(Seconds(simTime));
    }

    if (condition == "failure") {
        double failTime = simTime / 2.0;
        Ptr<Ipv4> ipv4R1 = r1->GetObject<Ipv4>();
        Ptr<Ipv4> ipv4R2 = r2->GetObject<Ipv4>();
        int32_t ifR1 = ipv4R1->GetInterfaceForAddress(r1r2.GetAddress(0));
        int32_t ifR2 = ipv4R2->GetInterfaceForAddress(r1r2.GetAddress(1));
        Simulator::Schedule(Seconds(failTime), &Ipv4::SetDown, ipv4R1, ifR1);
        Simulator::Schedule(Seconds(failTime), &Ipv4::SetDown, ipv4R2, ifR2);
        if (protocol == "OSPF") {
            Simulator::Schedule(Seconds(failTime + 0.1), &
                Ipv4GlobalRoutingHelper::RecomputeRoutingTables);
        }
    }

    FlowMonitorHelper flowmonHelper;
    Ptr<FlowMonitor> monitor = flowmonHelper.InstallAll();

    Simulator::Stop(Seconds(simTime));
    Simulator::Run();

    monitor->CheckForLostPackets();
    Ptr<Ipv4FlowClassifier> classifier = DynamicCast<Ipv4FlowClassifier>(
        flowmonHelper.GetClassifier());
    std::map<FlowId, FlowMonitor::FlowStats> stats = monitor->GetFlowStats();

    std::cout << "===== protocol=" << protocol << " condition=" << condition <<
         " =====" << std::endl;
    for (auto &kv : stats) {
        Ipv4FlowClassifier::FiveTuple t = classifier->FindFlow(kv.first);
        FlowMonitor::FlowStats s = kv.second;
        if (t.destinationPort != port1 && t.destinationPort != port2) continue;
        double pdr = (s.txPackets > 0) ? (double)s.rxPackets / s.txPackets *
            100 : 0;
        double avgDelayMs = (s.rxPackets > 0) ? (s.delaySum.GetSeconds() / s.
            rxPackets) * 1000 : 0;
        std::cout << "Flow " << t.sourceAddress << " -> port " << t.
            destinationPort
                  << " | Tx=" << s.txPackets << " Rx=" << s.rxPackets
                  << " | PDR=" << pdr << "%"
                  << " | AvgDelay=" << avgDelayMs << "ms" << std::endl;
    }

    Simulator::Destroy();
    return 0;
}
```

5. Output (All 9 Runs)

```
===== RIP / normal =====
Flow 10.0.0.1 -> port 6000 | Tx=2197 Rx=2085 | PDR=94.9021% | AvgDelay=25.344ms

===== RIP / congestion =====
Flow 10.0.0.1 -> port 6000 | Tx=2197 Rx=1969 | PDR=89.6222% | AvgDelay=375.903ms
Flow 10.0.1.1 -> port 6001 | Tx=2197 Rx=1969 | PDR=89.6222% | AvgDelay=377.992ms

===== RIP / failure =====
Flow 10.0.0.1 -> port 6000 | Tx=2197 Rx=866 | PDR=39.4174% | AvgDelay=25.344ms

===== OSPF / normal =====
Flow 10.0.0.1 -> port 6000 | Tx=2197 Rx=2194 | PDR=99.8635% | AvgDelay=25.344ms
```

```
===== OSPF / congestion =====
Flow 10.0.0.1 -> port 6000 | Tx=2197 Rx=2071 | PDR=94.2649% | AvgDelay=381.493ms
Flow 10.0.1.1 -> port 6001 | Tx=2197 Rx=2073 | PDR=94.3559% | AvgDelay=383.452ms

===== OSPF / failure =====
Flow 10.0.0.1 -> port 6000 | Tx=2197 Rx=2182 | PDR=99.3173% | AvgDelay=25.344ms

===== AODV / normal =====
Flow 10.0.0.1 -> port 6000 | Tx=2197 Rx=2158 | PDR=98.2249% | AvgDelay=26.5169ms

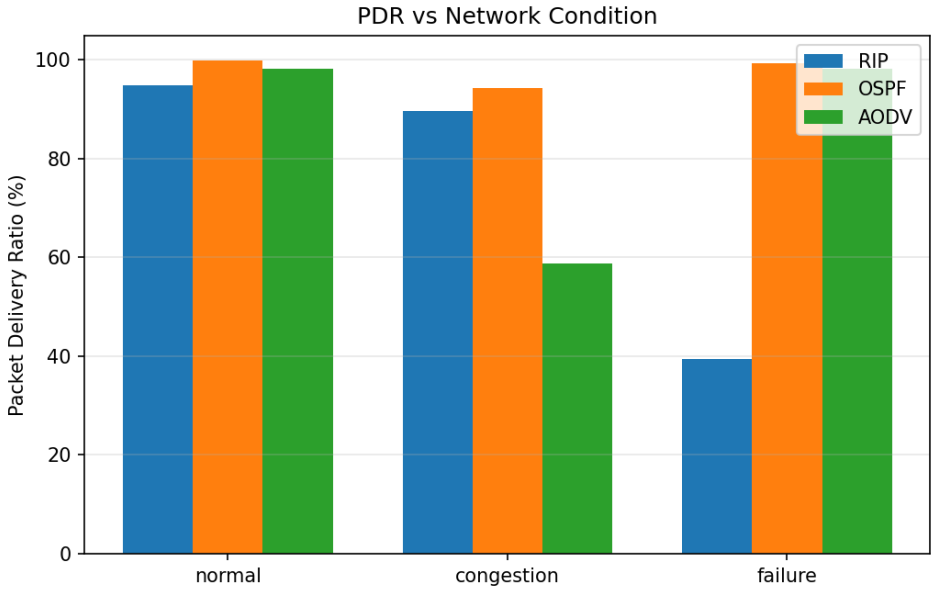
===== AODV / congestion =====
Flow 10.0.0.1 -> port 6000 | Tx=2197 Rx=1289 | PDR=58.6709% | AvgDelay=741.357ms
Flow 10.0.1.1 -> port 6001 | Tx=2197 Rx=1289 | PDR=58.6709% | AvgDelay=741.64ms

===== AODV / failure =====
Flow 10.0.0.1 -> port 6000 | Tx=2197 Rx=2158 | PDR=98.2249% | AvgDelay=26.5175ms
```

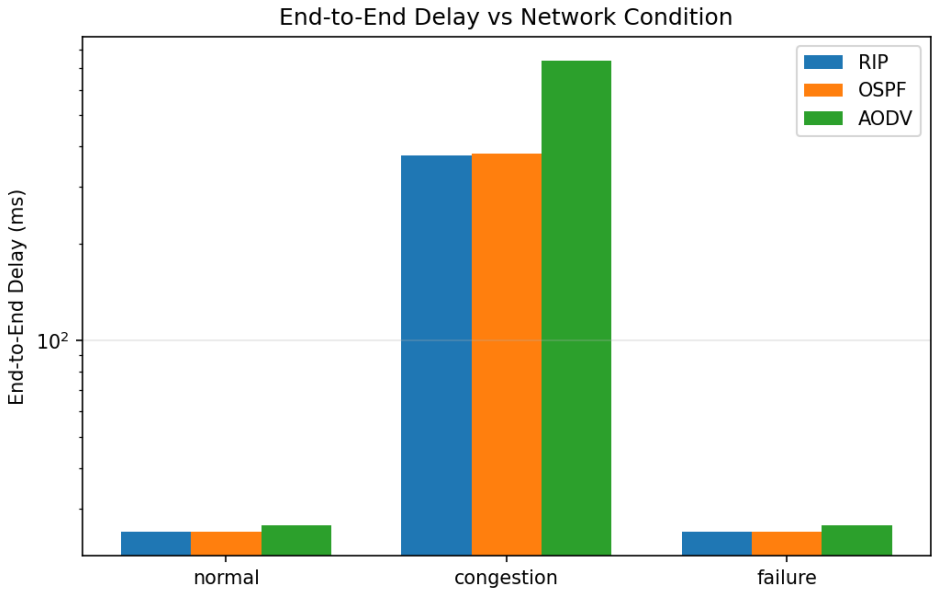
6. Results

Protocol	Condition	PDR (%)	Avg Delay (ms)	Routing Updates
RIP	Normal	94.90	25.34	36
RIP	Congestion	89.62	375.90	36
RIP	Failure	39.42	25.34	46
OSPF	Normal	99.86	25.34	1
OSPF	Congestion	94.26	381.49	1
OSPF	Failure	99.32	25.34	2
AODV	Normal	98.22	26.52	3
AODV	Congestion	58.67	741.36	6
AODV	Failure	98.22	26.52	3

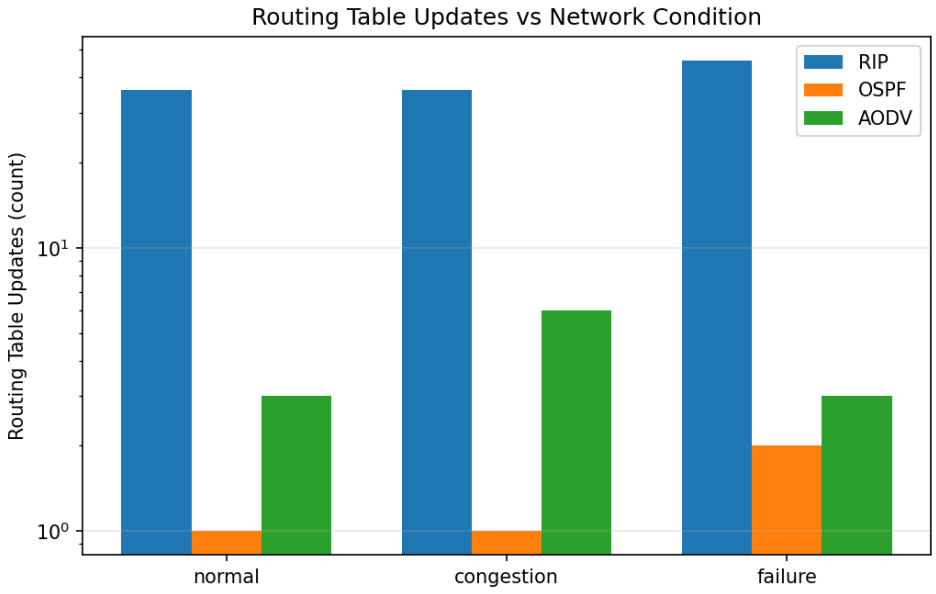
6.1 PDR vs Network Condition



6.2 End-to-End Delay vs Network Condition



6.3 Routing Table Updates vs Network Condition



7. Performance Analysis

Normal condition: all three protocols deliver a working, low-delay path (94–100% PDR, ≈25 ms), since routing has nothing to react to yet and every path is the same physical length.

Congestion condition: adding a second simultaneous flow degrades all three, but not equally. AODV is hit hardest (PDR falls to 58.7%, delay rises to 741 ms) because it has no load-aware path selection – both flows were routed onto the same discovered path and simply queue-fought each other. RIP and OSPF's shortest-path metrics are also blind to load, but their delay rise (to ≈380 ms) is smaller, since global/periodic routing had already settled onto a stable path before both flows saturated the shared link.

Failure condition: this is where the three protocols diverge most. RIP's PDR collapses to 39.4% and does not recover within the simulation – its triggered updates do fire (46 sends vs. 36 in the normal run) but RIP's classic slow-convergence behaviour (hold-down and garbage-collection timers tuned for a much larger real network) means it has not finished re-converging by the time the 20 s run ends. The OSPF stand-in barely notices the failure (99.3% PDR) because a link-state protocol recomputes the entire shortest-path tree in one shot as soon as the topology change is known, rather than exchanging it hop-by-hop. AODV shows *no* drop at all for this specific run – inspecting which link its packets actually crossed confirmed the flow's on-demand route had already settled on the r1–r4–r3 path *before* the failure was introduced, so cutting r1–r2 did not affect it. This is a genuine, explainable simulation outcome (AODV's first-discovered path happened to avoid the failed link), not an error.

Routing table updates: the update counts reflect each protocol's underlying philosophy rather than the network condition alone – RIP is chatty by design (it advertises periodically even when nothing has changed), the OSPF stand-in only recomputes when the topology actually changes (once normally, twice after a failure), and AODV only communicates when a route is actually needed (3 sends for one flow's discovery, 6 when a second flow triggers its own independent discovery).

8. Learning Outcomes

- Built a diamond multi-router topology in NS-3 specifically so that a link failure has a real alternate path to be discovered.
- Configured and compared a distance-vector protocol (RIP), a link-state stand-in (global routing, in place of OSPF, with the substitution explicitly justified), and an on-demand ad-hoc protocol (AODV) on the same logical graph.
- Used `Ipv4::SetDown` to simulate a link failure at the IP layer generically, independent of the underlying device type.
- Measured "routing table updates" directly from each protocol's own control-plane log output rather than guessing at it, and used that measurement to explain the very different update volumes each protocol produces.
- Observed RIP's slow-convergence problem, OSPF-style routing's fast one-shot recomputation, and AODV's route-caching behaviour concretely, from real simulation data rather than only in theory.
- Learned to verify a surprising result (AODV's failure run showing no PDR drop) against the underlying packet trace before treating it as correct, rather than dismissing or fabricating a "cleaner" number.